

the core in many users' neural network projects. While it can be applied for a great many things, getting started off is not that hard. TensorFlow has huge support online with no lack of example code to go through before you tackle your first project.

You can begin your TensorFlow journey by getting the Python library at <https://www.tensorflow.org/install/> and if you have a GPU that meets CUDA requirements, you can even optimise your execution using hardware acceleration.

Keras

Keras is an open source Python library which runs on top of other Python libraries such as TensorFlow, Microsoft Cognitive Toolkit, or Theano. Keras focuses on being more user-friendly, modular and extensible compared to the libraries that it runs on top of.

Keras has official support from Google's TensorFlow team (for the TensorFlow core library), allowing users to quickly build their code using the numerous implementations of commonly used neural networks methods already predefined in the Keras package. Like TensorFlow, Keras also supports hardware acceleration by the use of GPUs or TPUs, allowing the user far easier coding in almost the same execution time.

Unlike the other tools mentioned in our list, Keras is not a standalone library. Rather, it acts as a high-level API to other libraries that require comparatively more complex ways of coding. This allows users to achieve far more rapid prototyping with their code. As a trade-off for this level of user-friendly Python coding, you lose some level of control over your code. For example, in Keras, while you do have considerable control over your network, any of the lower level packages that implement Keras will control the finer things of the network.

This might lead you to wonder, why one should even use Keras? The

sole reason is that most of the time, while trying out new implementations of neural networks, that high level of control over the hyper parameters isn't required. The trade-off obtained by dropping that functionality makes far more sense to any programmer who only wants to see how the network performs. You can think of Keras as a way of getting the feel of what you are trying to build before trying to optimise it at a much lower level so that your networks perform the best. Keras has one of the most user-friendly approaches to coding neural networks. For anyone who just wants to try out the latest advances in current technology, Keras will almost always meet such a user's demands. One small shortcoming is that Keras is a pure Python library unlike TensorFlow.

Note: As of June 2019, Keras has officially been ported together with Tensorflow to become Tensorflow 2.0. This new version of Tensorflow cuts back on a lot of extras previously available, resulting in a more standardised approach of writing code using the Keras API. Tensorflow 2.0 is currently in beta stage, but it already implements a lot of the more commonly used features, so if you feel like foraging into the new and improved library, head over to: <https://www.tensorflow.org/beta>

It is possible for one to transform Keras models from Python to lower level languages such as C/C++.

As an example of how easy it is to understand and use Keras as compared to TensorFlow, let us take a look at both these libraries when used for the same application – the classification of handwritten numbers using the *Mnist* data set.

First let us look at an example of using pure TensorFlow to finish the task, as shown below:

```
import tensorflow as tf
mnist = tf.keras.datasets.mnist
```

```
n_nodes = 512
n_classes = 10
batch_size = 100

x = tf.placeholder('float', [None, 784])
y = tf.placeholder('float')

def neural_network_model(data):
    layer_1 = {'weights':tf.
Variable(tf.random_normal([784, n_
nodes])),
                'biases':tf.
Variable(tf.random_normal([n_nodes]))}
    output_layer = {'weights':tf.
Variable(tf.random_normal([n_nodes,
n_classes])),
                'biases':tf.
Variable(tf.random_normal([n_
classes])),}
    l1 = tf.add(tf.matmul(data,layer_1[
'weights']], layer_1['biases'])
    l1 = tf.nn.relu(l1)
    output = tf.matmul(l1,output_
layer['weights']) + output_
layer['biases']
    return output

def train_neural_network(x):
    prediction = neural_network_
model(x)
    cost = tf.reduce_mean( tf.nn.
sparse_softmax_cross_entropy_with_logit
s(logits=prediction,labels=y) )
    optimizer = tf.train.
AdamOptimizer().minimize(cost)
    hm_epochs = 5
    with tf.Session() as sess:
        sess.run(tf.initialize_all_
variables())
        for epoch in range(hm_epochs):
            epoch_loss = 0
            for _ in range(int(mnist.
train.num_examples/batch_size)):
                epoch_x, epoch_y =
mnist.train.next_batch(batch_size)
                _, c = sess.
run([optimizer, cost], feed_dict={x:
epoch_x, y: epoch_y})
                epoch_loss += c
            print('Epoch',
epoch, 'completed out of',hm_
epochs,'loss:',epoch_loss)
```